

```
!pip install keras-tuner
```

```
import tensorflow as tf
from tensorflow.keras.datasets import imdb
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense, Dropout
import keras_tuner as kt
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import confusion_matrix
```

```
Requirement already satisfied: keras-tuner in /usr/local/lib/python3.12/dist-packages (1.4.8)
Requirement already satisfied: keras in /usr/local/lib/python3.12/dist-packages (from keras-tuner) (3.13.2)
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages (from keras-tuner) (26.0)
Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-packages (from keras-tuner) (2.32.4)
Requirement already satisfied: kt-legacy in /usr/local/lib/python3.12/dist-packages (from keras-tuner) (1.0.5)
Requirement already satisfied: grpcio in /usr/local/lib/python3.12/dist-packages (from keras-tuner) (1.80.0)
Requirement already satisfied: protobuf in /usr/local/lib/python3.12/dist-packages (from keras-tuner) (5.29.6)
Requirement already satisfied: typing-extensions~=4.12 in /usr/local/lib/python3.12/dist-packages (from grpcio->keras-tuner) (4.12.0)
Requirement already satisfied: absl-py in /usr/local/lib/python3.12/dist-packages (from keras->keras-tuner) (1.4.0)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (from keras->keras-tuner) (2.0.2)
Requirement already satisfied: rich in /usr/local/lib/python3.12/dist-packages (from keras->keras-tuner) (13.9.4)
Requirement already satisfied: namex in /usr/local/lib/python3.12/dist-packages (from keras->keras-tuner) (0.1.0)
Requirement already satisfied: h5py in /usr/local/lib/python3.12/dist-packages (from keras->keras-tuner) (3.16.0)
Requirement already satisfied: optree in /usr/local/lib/python3.12/dist-packages (from keras->keras-tuner) (0.19.0)
Requirement already satisfied: ml-dtypes in /usr/local/lib/python3.12/dist-packages (from keras->keras-tuner) (0.5.4)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests->keras-tuner) (3.3.2)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests->keras-tuner) (3.11)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests->keras-tuner) (2.5.0)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests->keras-tuner) (2026.0.2)
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.12/dist-packages (from rich->keras->keras-tuner) (3.0.0)
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3.12/dist-packages (from rich->keras->keras-tuner) (2.18.0)
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.12/dist-packages (from markdown-it-py>=2.2.0->rich->keras->k
```

```
MAX_FEATURES = 10000 # Vocabulary size
MAX_LEN = 200 # Sequence length
```

```
print("Loading data...")
```

```
(x_train, y_train), (x_test, y_test) = imdb.load_data(num_words=MAX_FEATURES)
```

```
print("Train samples:", len(x_train))
print("Test samples:", len(x_test))
```

```
# Padding
x_train = pad_sequences(x_train, maxlen=MAX_LEN)
x_test = pad_sequences(x_test, maxlen=MAX_LEN)
```

```
print("Shape after padding:", x_train.shape)
```

```
Loading data...
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/imdb.npz
17464789/17464789 ————— 0s 0us/step
Train samples: 25000
Test samples: 25000
Shape after padding: (25000, 200)
```

```
def build_model(hp):
    model = Sequential()

    # Embedding layer
    model.add(Embedding(
        input_dim=MAX_FEATURES,
        output_dim=hp.Int('embedding_dim', 32, 128, step=32),
        input_length=MAX_LEN
    ))

    # LSTM layer
    model.add(LSTM(
        units=hp.Int('lstm_units', 32, 128, step=32)
    ))

    # Dropout
    model.add(Dropout(
```

```

        rate=hp.Float('dropout_rate', 0.2, 0.5, step=0.1)
    ))

    # Output
    model.add(Dense(1, activation='sigmoid'))

    model.compile(
        optimizer=tf.keras.optimizers.Adam(
            hp.Choice('learning_rate', [1e-2, 1e-3, 1e-4])
        ),
        loss='binary_crossentropy',
        metrics=['accuracy']
    )

    return model

```

```

tuner = kt.RandomSearch(
    build_model,
    objective='val_accuracy',
    max_trials=5,
    directory='tuner_dir',
    project_name='imdb_lstm'
)

print("Starting hyperparameter tuning...")

tuner.search(
    x_train, y_train,
    epochs=3,
    validation_split=0.2,
    batch_size=64
)

best_hp = tuner.get_best_hyperparameters(1)[0]

print("\nBest Hyperparameters:")
print("Embedding Dim:", best_hp.get('embedding_dim'))
print("LSTM Units:", best_hp.get('lstm_units'))
print("Dropout Rate:", best_hp.get('dropout_rate'))
print("Learning Rate:", best_hp.get('learning_rate'))

```

```

Trial 5 Complete [00h 05m 26s]
val_accuracy: 0.8740000128746033

Best val_accuracy So Far: 0.8870000243186951
Total elapsed time: 00h 23m 32s

Best Hyperparameters:
Embedding Dim: 64
LSTM Units: 128
Dropout Rate: 0.30000000000000004
Learning Rate: 0.01

```

```
model = tuner.hypermodel.build(best_hp)
```

```

history = model.fit(
    x_train, y_train,
    epochs=7,
    validation_split=0.2,
    batch_size=64
)

```

```

Epoch 1/7
313/313 ————— 150s 468ms/step - accuracy: 0.7574 - loss: 0.4928 - val_accuracy: 0.8716 - val_loss: 0.3100
Epoch 2/7
313/313 ————— 148s 473ms/step - accuracy: 0.9032 - loss: 0.2417 - val_accuracy: 0.8828 - val_loss: 0.3018
Epoch 3/7
313/313 ————— 136s 436ms/step - accuracy: 0.9434 - loss: 0.1539 - val_accuracy: 0.8676 - val_loss: 0.3513
Epoch 4/7
313/313 ————— 136s 435ms/step - accuracy: 0.9609 - loss: 0.1116 - val_accuracy: 0.8708 - val_loss: 0.4094
Epoch 5/7
313/313 ————— 137s 438ms/step - accuracy: 0.9742 - loss: 0.0756 - val_accuracy: 0.8654 - val_loss: 0.4604
Epoch 6/7
313/313 ————— 137s 436ms/step - accuracy: 0.9765 - loss: 0.0672 - val_accuracy: 0.8582 - val_loss: 0.5084
Epoch 7/7
313/313 ————— 145s 447ms/step - accuracy: 0.9801 - loss: 0.0584 - val_accuracy: 0.8552 - val_loss: 0.5524

```

```

acc = history.history['accuracy']
val_acc = history.history['val_accuracy']
loss = history.history['loss']
val_loss = history.history['val_loss']

epochs = range(1, len(acc) + 1)

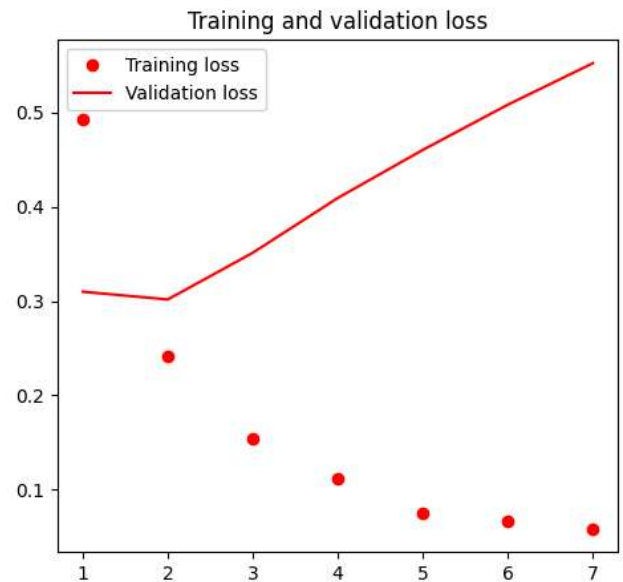
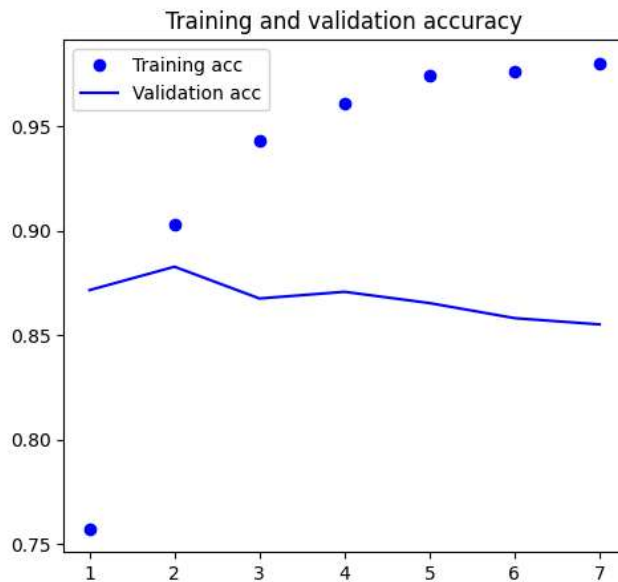
plt.figure(figsize=(12,5))

# Accuracy
plt.subplot(1,2,1)
plt.plot(epochs, acc, 'bo', label='Training acc')
plt.plot(epochs, val_acc, 'b', label='Validation acc')
plt.title('Training and validation accuracy')
plt.legend()

# Loss
plt.subplot(1,2,2)
plt.plot(epochs, loss, 'ro', label='Training loss')
plt.plot(epochs, val_loss, 'r', label='Validation loss')
plt.title('Training and validation loss')
plt.legend()

plt.show()

```



```

print("\nEvaluating on test data...")

test_loss, test_acc = model.evaluate(x_test, y_test)

print(f"Test Accuracy: {test_acc * 100:.2f}%")
print(f"Test Loss: {test_loss:.4f}")

```

```

Evaluating on test data...
782/782 ————— 87s 111ms/step - accuracy: 0.8548 - loss: 0.5599
Test Accuracy: 85.48%
Test Loss: 0.5599

```

```

word_index = imdb.get_word_index()

def predict_sentiment(text):
    tokens = [word_index.get(word, 0) for word in text.lower().split()]
    padded = pad_sequences([tokens], maxlen=MAX_LEN)
    prediction = model.predict(padded)[0][0]

    print(f"\nText: {text}")
    print(f"Score: {prediction:.4f}")

    if prediction > 0.5:
        print("Sentiment: POSITIVE")
    else:
        print("Sentiment: NEGATIVE")

```

```
predict_sentiment("This movie was amazing and thrilling")
predict_sentiment("I hated this movie it was terrible")
```

Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/imdb_word_index.json

1641221/1641221 — 0s 0us/step

1/1 — 0s 246ms/step

Text: This movie was amazing and thrilling

Score: 0.9666

Sentiment: POSITIVE

1/1 — 0s 116ms/step

Text: I hated this movie it was terrible

Score: 0.4770

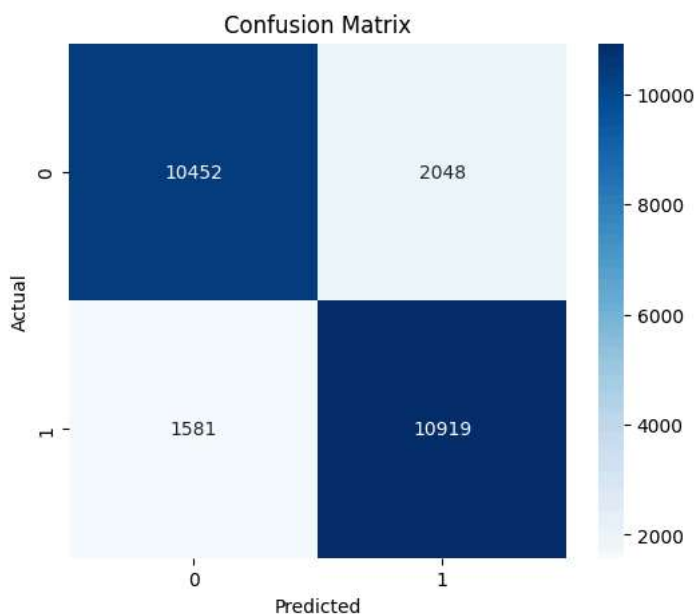
Sentiment: NEGATIVE

```
y_pred_probs = model.predict(x_test)
y_pred = (y_pred_probs > 0.5).astype("int32")

cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(6,5))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()
```

WARNING:tensorflow:5 out of the last 35 calls to <function TensorFlowTrainer.make_predict_function.<locals>.one_step_on_data_dis
782/782 — 91s 116ms/step



```
plt.figure(figsize=(8,5))
plt.hist(y_pred_probs, bins=50, alpha=0.7)
plt.axvline(x=0.5)
plt.title("Prediction Probability Distribution")
plt.xlabel("Probability")
plt.ylabel("Count")
plt.show()
```

